	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): 8

Integrante(s): 323078401

323115296

323082392

323139441

323253916

No. de lista o brigada: 04

Semestre: 2027-I

Fecha de entrega: 23/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**

1.1. Planteamiento del problema 2

1.2. Motivación 2

1.3. Objetivos 2

2. Marco Teórico **2**

3. Desarrollo **3**

3.1. Actividad 1: Análisis de las respuestas de las LLMs 3

3.2. Actividad 2: Programa para calcular la huella de carbono 6

4. Resultados **7**

4.1. Resultados de la Práctica 8 7

5. Conclusiones **10**

6. Referencias **10**

1. Introducción

1.1 Planteamiento del problema

El problema consiste en modelar tres objetos no vinculados por herencia: un edificio, un automóvil y una bicicleta, para agruparlas en una misma colección y calcular su huella de carbono respectiva. A la par, preguntar a LLMs sobre la interfaz gráfica de usuario (GUI) orientada a la autenticación de usuarios en Java.

1.2 Motivación

La comprensión de las interfaces y del polimorfismo dinámico en lugar de limitarnos a la reutilización de código con herencia simple; y el uso de estructuras de datos polimórficas para que los objetos interactúen de forma estandarizada. Analizar cómo interactúan los componentes visuales, el manejo de eventos en interfaces gráficas y las restricciones de seguridad en memoria.

1.3 Objetivos

- Implementar tres clases independientes que tengan en común el uso de una interfaz para calcular la huella de carbono de cada uno.
- Comparar las respuestas generadas por distintos LLMs al solicitarles como crear una interfaz gráfica de inicio de sesión.
- Comprender los principios del polimorfismo dinámico e interfaces.

2. Marco Teórico

Una **interfaz** representa un contrato formal que especifica qué comportamiento debe exhibir una clase sin predeterminar los detalles de su implementación [1]. A diferencia de la herencia de clases, donde se impone una relación estricta del tipo ńIS-Aż, las interfaces permiten establecer una relación de capacidad o comportamiento común. Al implementar una interfaz mediante la palabra reservada correspondiente, la clase asume la obligación de proveer cuerpo a los métodos declarados, garantizando la compatibilidad con cualquier componente del sistema que dependa de dicha abstracción [1].

El **polimorfismo** dinámico es el mecanismo por el cual una llamada a un método se vincula con su implementación correspondiente en tiempo de ejecución, en función del tipo real de la instancia que la ejecuta y no de la referencia [2]. Esto hace posible manipular instancias de diversas clases derivadas a través de una misma variable de referencia del tipo de la interfaz. Para gestionar estos conjuntos de objetos en memoria de forma flexible, el entorno de Java proporciona el marco de colecciones (*Java Collections Framework*), del cual, la clase **ArrayList**, implementa una lista redimensionable sobre la base de un arreglo dinámico [3].

Por otro lado, la interacción gráfica con el usuario implementa la programación dirigida por eventos (*Event-Driven Programming*) [4]. En este modelo, el flujo de ejecución no sigue una secuencia lineal, sino que reacciona a las acciones realizadas por el operador humano sobre la interfaz, tales como clics de ratón o pulsaciones de teclado. En Java, las interfaces gráficas se han articulado mediante bibliotecas como **Swing** y **JavaFX** [4], [5]. Swing está integrada nativamente en el paquete de escritorio del compilador [4]. JavaFX es una alternativa moderna [5].

Dentro de la arquitectura de interfaces de usuario, la recepción y procesamiento de eventos se gestiona mediante el patrón de diseño Observador a través de interfaces escuchadoras (*listeners*) como **ActionListener** [6]. Cuando un control emite un evento, el despachador de la interfaz notifica al escuchador, ejecutando la lógica asociada [4]. Asimismo, el manejo de credenciales dentro de formularios de autenticación involucra la seguridad en memoria. El empleo de componentes dedicados a contraseñas (**JPasswordField** o **PasswordField**) prioriza la recuperación de datos mediante arreglos mutables de caracteres (**char[]**) en lugar de objetos inmutables de cadena de texto (**String**), permitiendo la sobreescritura deliberada de la memoria volátil inmediatamente después de la validación y evitando seguir teniendo de forma indefinida datos sensibles [4].

3. Desarrollo

3.1 Actividad 1: Análisis de las respuestas de las LLMs

La primera actividad consistió en plantear a diferentes LLMs la elaboración de una interfaz gráfica de usuario orientada a la autenticación de acceso (usuario y contraseña) en Java.

Análisis de la Respuesta 1

En la primera consulta se solicitó explicar y proporcionar un código funcional, detallar el uso de librerías restringiendose a JavaFX o AWT. En la respuesta, el modelo seleccionó JavaFX y relegó AWT a un cuadro comparativo final.

La solución estructuró la ventana principal extendiendo la clase base de aplicación, organizó los controles verticalmente y asoció la validación a una función lambda vinculada al botón principal, presentando retroalimentación por medio de cuadros de diálogo emergentes. Cumplio con la solicitud de explicar detalladamente las clases importadas, incorporando instrucciones de compilación modular desde la terminal.

Sin embargo, recurrió a una validación con credenciales fijas y empleó cadenas de texto para la recuperación de la contraseña, dejando de lado consideraciones de seguridad de memoria que se vinculan al uso de campos de texto oculto en aplicaciones de escritorio.

Análisis de la Respuesta 2

La segunda consulta se formuló mediante una instrucción directa pidiendo únicamente cómo elaborar la interfaz de inicio de sesión en Java sin fijar librerías ni condiciones de diseño.

Ante la ausencia de restricciones, el modelo optó por emplear Swing y AWT, argumentando su portabilidad nativa sin dependencias externas. La solución construyó un marco gráfico centrado con un administrador de diseño bidimensional avanzado, programó la captura de la contraseña mediante un arreglo de caracteres e implementó una limpieza manual de dicho arreglo sobrescribiéndolo con caracteres nulos tras la verificación.

Asimismo, la respuesta contempló la adaptación a la apariencia del sistema operativo. La falta de especificaciones del prompt provocó que la herramienta asumiera decisiones técnicas complejas de forma autónoma, entregando una implementación funcional pero con escasa teoría sobre las clases utilizadas.

Análisis de la Respuesta 3

La tercera consulta se diseñó mediante una formulación estructurada y técnica, asignando al modelo el rol de desarrollador experimentado e imponiendo requisitos concretos: justificación del marco gráfico, inclusión de elementos auxiliares de interfaz: botones secundarios, alternancia visual de la contraseña mediante un conmutador y área fija de mensajes de estado en la ventana en lugar de cuadros emergentes, desacoplamiento

arquitectónico y manejo riguroso de contraseñas en memoria volátil.

El modelo respondió adoptando el patrón arquitectónico Modelo-Vista-Controlador dentro de un único archivo ejecutable. En la vista se integraron componentes para manipular dinámicamente el carácter de enmascaramiento del campo de contraseña y etiquetas con colores semánticos para desplegar advertencias. En el modelo se implementó la validación temporal de credenciales, y en el controlador se aseguraron bloques de control con cláusulas de liberación en los que los arreglos de caracteres fueron borrados obligatoriamente tras la verificación. La rigurosidad del prompt condicionó al modelo a entregar una solución modular y apegada a buenas prácticas de ingeniería de software.

Análisis de la Respuesta 4

La cuarta consulta orientó su planteamiento hacia una explicación conceptual, teórica y de herramientas de desarrollo, solicitando los criterios para elegir entre bibliotecas gráficas, la lógica interna y las herramientas necesarias, sin demandar un bloque de código ejecutable.

La respuesta de este modelo aportó un enfoque formativo, clasificando las ventajas y limitaciones operativas de Swing, JavaFX y AWT. Además, contextualizó el desarrollo integrando herramientas de soporte como extensiones para editores de código, el modelado visual con programas auxiliares y la compatibilidad de ventanas gráficas sobre subsistemas de capas operativas. Para clarificar la lógica de control, el modelo integró diagramas de máquina de estados y simuladores de transición del flujo de autenticación, ejemplificando el ciclo de vida de las ventanas y la liberación de memoria. Aunque esta interacción no ofreció un código listo para compilar, proporcionó el marco conceptual más completo sobre la interacción entre la interfaz y la lógica.

Observaciones

Al contrastar las cuatro interacciones se observó que la amplitud y el nivel técnico del prompt determinaron directamente el enfoque del resultado. Las instrucciones excesivamente abiertas de la segunda consulta dan pie a soluciones viables pero dependientes del criterio arbitrario de la IA, mientras que las especificaciones con restricciones explícitas de diseño y seguridad indujeron la entrega de código desacoplado y robusto. Del mismo modo, en la cuarta consulta no se exige código, por lo que los modelos se orientan hacia lo conceptual y la arquitectura del sistema.

3.2 Actividad 2: Programa para calcular la huella de carbono

La segunda actividad se enfocó en un programa para calcular la huella de carbono de tres entidades diferentes: un edificio, un automóvil y una bicicleta.

Se diseñaron tres clases independientes que carecen de una relación de herencia entre sí. Para asegurar que cada una modelara su comportamiento particular, se les asignaron atributos y métodos exclusivos. La clase correspondiente al automóvil encapsuló los kilómetros recorridos y el rendimiento de combustible por litro, implementando como comportamiento propio un método para calcular el volumen total de litros consumidos a partir de la división entre ambas magnitudes. La clase de la bicicleta incorporó la distancia transitada y el material de manufactura del cuadro, incluyendo una operación exclusiva destinada a mostrar el ajuste de la presión neumática según el tipo de bastidor. Por su parte, la clase del edificio definió su superficie total en metros cuadrados, el número de pisos y el consumo eléctrico anual por unidad de superficie, añadiendo un método particular para determinar el área promedio construida por cada nivel.

Para unificar la interacción con estos elementos, se declaró la interfaz denominada **HuellaCarbono** que estableció el método de consulta **getHuellaCarbono()**.

Cada una de las tres clases implementó la interfaz adaptando el cálculo de la huella de carbono. En el automóvil, el método calcula los litros consumidos y los multiplica por un factor fijo de emisión derivado de la combustión de hidrocarburos, tomándolo como 2.31 de kilogramos de dióxido de carbono por litro. En la bicicleta, donde las emisiones no provienen de combustible sino del proceso de extracción y manufactura amortizado a lo largo de su vida útil, el método evalúa el material del cuadro mediante una estructura de selección múltiple sobre la cadena convertida a minúsculas, asignando factores por kilómetro para fibra de carbono, aluminio y acero. En el edificio, el método determina la demanda energética global multiplicando la superficie construida por la tasa unitaria de consumo, para posteriormente aplicar un factor de intensidad de carbono por red eléctrica de 0.45 kilogramos por kilovatio-hora. Asimismo, cada entidad sobrescribió el método toString para generar una descripción propia de sus parámetros.

En la clase Main, dentro del método de entrada, se instanciaron los tres objetos asignando valores directos de prueba: una superficie de 2500 metros cuadrados con 5 niveles y 110 kilovatios-hora por metro cuadrado para el edificio; 12000 kilómetros con un rendimiento de 14.5 kilómetros por litro para el vehículo motorizado; y 3200 kilómetros con cuadro de aluminio para la bicicleta. Las tres referencias se almacenaron secuencialmente dentro de una lista genérica parametrizada con el tipo de la interfaz.

Finalmente, el programa recorrió la colección mediante un for each. En cada

ciclo, la variable de control de tipo interfaz invocó de manera implícita la representación en cadena del objeto y, posteriormente, hizo la llamada al método de cálculo de huella de carbono. La máquina virtual de Java resolvió en tiempo de ejecución la vinculación dinámica correspondiente a cada clase concreta, mostrando las huellas de carbono resultantes con dos decimales e imprimiendolo.

4. Resultados

4.1 Resultados de la Práctica 8

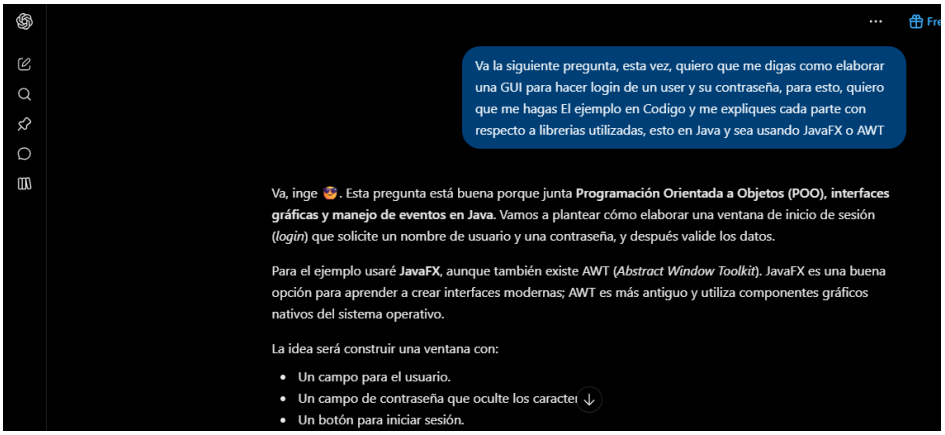


Figura 1: Respuesta 1 de la IA generativa.

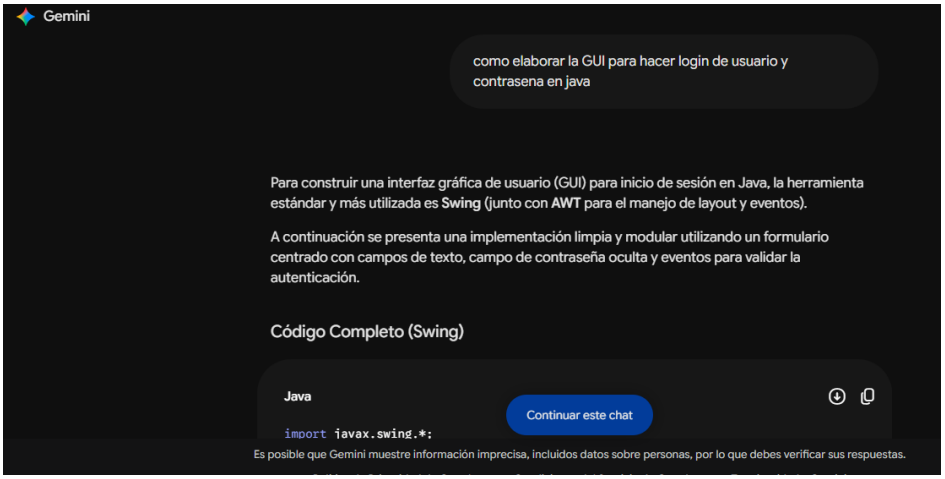


Figura 2: Respuesta 2 de la IA generativa.

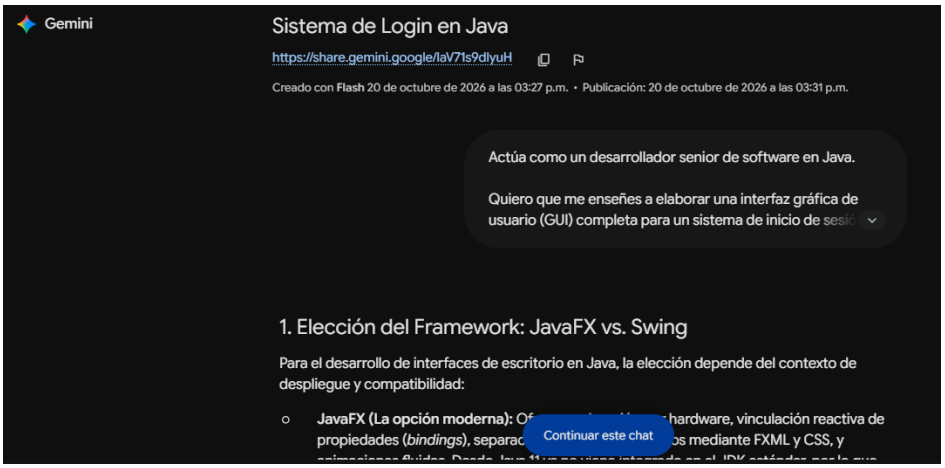


Figura 3: Respuesta 3 de la IA generativa.

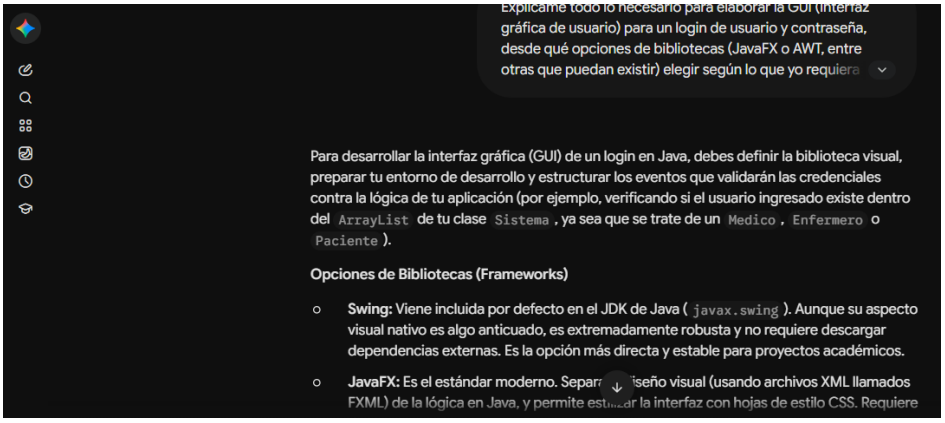


Figura 4: Respuesta 4 de la IA generativa.

Resultados de ejecución

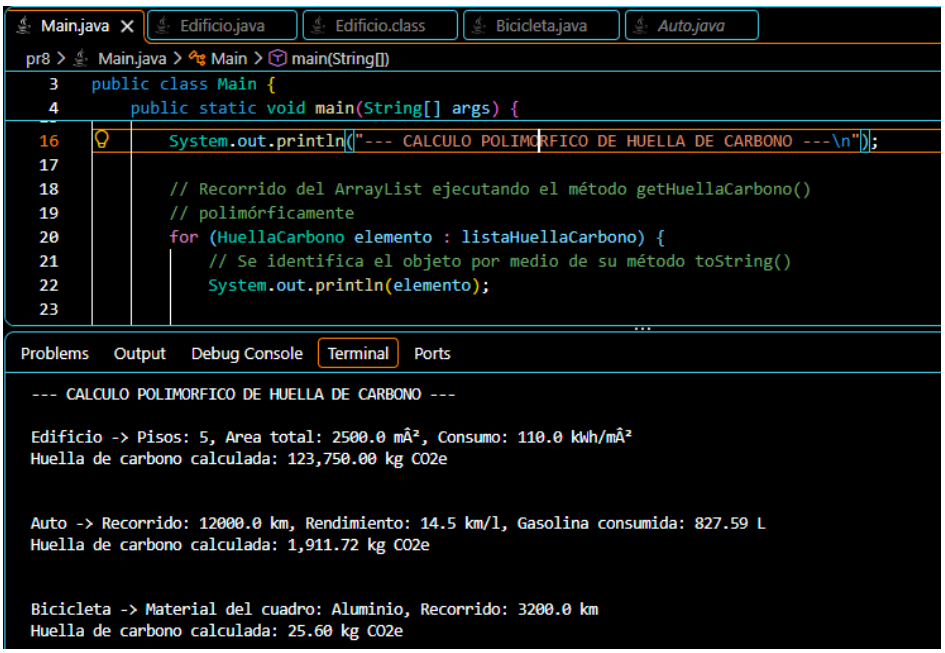


Figura 5: Compilación del programa de huella de carbono.

Diagrama 01: Diagrama de Clases UML

Ver imagen 01_Diagrama_Clases_UML.png en GitHub

Diagrama 02: Clase Auto

Ver imagen 02_Clase_Auto.png en GitHub

Diagrama 03: Clase Bicicleta

Ver imagen 03_Clase_Bicicleta.png en GitHub

Diagrama 04: Clase Edificio

Ver imagen 04_Clase_Edificio.png en GitHub

Diagrama 05: Clase Main

Ver imagen 05_Clase_Main.png en GitHub

5. Conclusiones

Los resultados obtenidos en la práctica mostraron la efectividad del uso de interfaces para hacer uso de clases que no comparten un vínculo dentro de la jerarquía de herencia. Al implementar una interfaz común entre las entidades se logró unificar el tratamiento de sus emisiones ambientales bajo una estructura de colección única, delegando en la máquina virtual de Java la resolución dinámica del cálculo correspondiente a cada objeto en tiempo de ejecución. Mientras que las consultas mostraron varios elementos relevantes aplicados para las interfaces gráficas como: manejo seguro de tipos de datos en memoria volátil, programación dirigida por eventos e interfaces escuchadoras (listeners).

6. Referencias

[1] Oracle, Ïnterfaces,"*The Java Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html

[2] Oracle, "Polymorphism,"*The Java Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html

[3] Oracle, "Lesson: Introduction to Collections,"*The Java Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/collections/intro/index.html

[4] Oracle, Çreating a GUI With Swing,"*The Java Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/

[5] OpenJFX, "JavaFX Documentation Project,.ºpenJFX Community. [En línea]. Disponible en: https://openjfx.io/

[6] Oracle, "Writing Event Listeners,"*The Java Tutorials*, Oracle Corporation. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/events/index.html